

POCKET SMART AI – YOUR SMART BUDGET & RECOMMENDATION ASSISTANT

Pocket smart AI - Your smart Budget & Recommendation Assistant

Definition :

Pocket Smart AI is an intelligent, user-centric financial management and lifestyle recommendation platform designed to automate budget tracking and simplify personal expense planning. The platform leverages advanced machine learning algorithms and Large Language Models (LLMs) to seamlessly categorize daily expenses, analyze individual spending habits, and forecast future financial trends.

Team Members Name & Roles:

1.Name:Sanjana S(Team Leader)

Role: Coding

2.Name: Soniya sree S(Team Member)

Role: API Testing video

3.Name: Thasbeena A(Team Member)

Role: Documentation

4.Name:Ragavi D(Team Member)

Role: Voice over

5.Name: Dhikshitha S(Team Member)

Role: Demo video

1.0 Executive Summary & Platform Purpose:

Pocket Smart AI is a scalable, cloud-ready RESTful backend platform engineered using Node.js and Express.js, leveraging MongoDB and Mongoose ODM for operational data persistence.

1.1 Core Objectives:

- Context-Aware Insights: Convert traditional numerical financial metrics (income, transaction histories, monthly bills, subscription fees) into actionable, localized recommendations.
- AI Orchestration: Seamlessly integrate Google Gemini AI (via the @google/generative-ai SDK) to synthesize natural language daily advice for savings, smart shopping, and investment safety.
- State Persistence: Securely persist user profiles, encrypted credentials, and curated budget goals across mobile and web client interfaces.
- Fault-Tolerant Resilience: Implement automatic fallback mocking mechanisms to ensure API responses remain continuous during upstream service outages or missing third-party keys.



1.2 High-Level Platform Capabilities:

Key Architectural Features:

1.JWT Access Guarding: Enforces stateless, token-based authentication with bcrypt password encryption.

2.Relational Document Mapping: Utilizes Mongoose schemas with indexed foreign references and unique multi-key constraints.

3.1. Centralized Middleware: Intercepts request flows for header evaluation, expense request sanitization, and structured financial error logging.

2.0 Scenario-Based Case Study & Requirements:

2.1 Background Story:

Alex is a busy freelance professional and small business owner who manages multiple distinct income streams and recurring monthly expenses. Alex requires a unified application interface capable of tracking real-time financial logs across several favorite spending categories simultaneously, while avoiding the overhead of analyzing complex economic charts and balance sheets manually.

2.2 Problem Formulation:

Pocket Smart AI – Friction Point Analysis		
CHALLENGE AREA	OBSERVED FRICTION POINT	IMPACT ON USER EXPERIENCE
Data Overload	Raw numerical transaction feeds (e.g., "\$120 on utilities, \$45 on dining, 15%") lack human-centric context.	Users spend unnecessary effort calculating what lifestyle cuts or strategies to implement.
Multi-Account Sync	Lack of centralized preference and balance storage across mobile and web clients.	Frequent re-entry of budget goals and transaction logs leads to user frustration.
Stateless Security	Inconsistent ledger protection across external network access points.	Risk of transaction tampering or compromised financial user context.

2.3 Software & Hardware Environment Requirements:

Software Specifications:

- Operating System: Cross-platform (Windows 10/11, macOS Monterey+, Linux Ubuntu 20.04+ LTS).
- Runtime Engine: Node.js (v16.0.0+ LTS recommended).
- Package Controller: npm (v8.0.0+) for dependency tree management.
- Database Engine: MongoDB (v5.0+ Community/Enterprise Edition).
- SDK Dependencies: @google/generative-ai, express, mongoose, jsonwebtoken, bcryptjs.

3.0 System Architecture & Design Patterns:

The Pocket Smart AI application is constructed around a multi-layered Model-View-Controller (MVC) framework, adapted specifically for headless API services. The design strictly isolates budget business logic, transactional data processing, and financial database mapping.

3.1 Architecture Blueprint:

Component Breakdown

Layer	Component Name	Primary Responsibilities
Interface	API Gateway / Client	Receives HTTP requests (GET, POST, PUT, DELETE) from Postman, Web UI, or Mobile apps.
Routing	Express Router Shields	Evaluates URL pathways, checks JWT authorization headers, and routes financial payloads to controllers.
Controller	Execution Controllers	Orchestrates business workflows, validates budget parameters, and coordinates service calls.
Service	External Adapters	Handles isolated logic for financial data normalization and Google Gemini AI budget prompting.
Model	Mongoose ODM	Validates schemas, applies transaction defaults, and enforces primary/foreign key rules on MongoDB.

3.2 Architectural Flow:

1. Client submits request with JSON payload containing financial transactions or budget updates to a protected route endpoint.
2. authMiddleware verifies Bearer JWT validity to protect sensitive user ledger records.
3. Target Controller receives the validated request and dispatches the data payload to the BudgetService or AIService.
4. Service layer queries transaction logs via Mongoose Models or makes external HTTPS requests to the Gemini AI orchestration engine.

Formatted JSON array with categorized metrics or conversational lifestyle recommendations is returned to the client with appropriate HTTP status codes (200, 201, 400, 401, 500).

4.0Data Architecture & ER Modeling:

To guarantee strict enterprise structure inside our hybrid document schema framework, MongoDB utilizes explicitly defined Mongoose models. Below are the production-grade schema declarations mapping our core database collections:

Java script:

```
// 1. USERS COLLECTION SCHEMA DEFINITION
```

```
const UserSchema = new mongoose.Schema({  
  username: {  
    type: String,  
    required: [true, "Username is required"],  
    unique: true,  
    trim: true,  
    minlength: [3, "Username must be at least 3 characters  
long"]  
  },  
  email: {  
    type: String,  
    required: [true, "Email verification target is required"],  
    unique: true,  
    lowercase: true,  
    trim: true,  
    match: [/^\\w+([\\.-]?\\w+)*@\\w+([\\.-]?\\w+)*(\\.\\w{2,3})+$/,  
"Please fill a valid email address"]  
  },
```

```
passwordHash: {  
  type: String,  
  required: [true, "Secure cryptographic hash string is  
required"]  
},  
accountTier: {  
  type: String,  
  enum: ["Standard", "Premium", "Enterprise"],  
  default: "Standard"  
},  
createdAt: {  
  type: Date,  
  default: Date.now  
}  
, { timestamps: true });
```

```
// Ensure optimized lookup index profiles on unique  
authentication fields
```

```
UserSchema.index({ email: 1, username: 1 });
```



4.1 Entity Relationship Diagram Structure:

Primary Collections & Relationships:

- **USERS Collection:** Serves as the primary account authority. Holds user identity credentials, security credentials, and creation timestamps.
- **SAVED_LOCATIONS Collection:** Holds user-defined location parameters including city name, country code, and geographic coordinates. Contains a mandatory foreign key reference (`user_id`) mapping directly back to the **USERS** collection.
- **WEATHER DATA Collection (Extended Model):** Caches live weather metrics (temperature, feels like, humidity, environmental condition) indexed dynamically by location.
- **AI_INSIGHTS Collection (Extended Model):** Stores the generated Google Gemini AI prompt outputs (personalized suggestions/budgets) mapped directly to matching weather instances.

4.2 Entity Mapping Matrix:

SOURCE ENTITY	TARGET ENTITY	CARDINALITY	REFERENTIAL CONSTRAINT
User	Location	One-to-Many (1: N)	User deletion or query filter isolates child saved locations.
Location	WeatherData	One-to-Many (1: N)	Location primary key links atmospheric query historical logs.
WeatherData	AI_Insight	One-to-One (1: 1)	Each distinct weather state references an AI recommendation block.

5.0Executive Summary & System Vision

ocket Smart AI is a next-generation mobile and edge-intelligence application designed to deliver real-time, context-aware assistance coupled with personalized, AI-generated daily productivity and lifestyle recommendations. By integrating device telemetry (location, motion, user activity) with cloud-based generative AI models, the platform transforms raw physical and behavioral data into actionable, immediate advice for users on the go.

5.1 Key System Attributes

- Sub-Second Latency: Lightweight on-device processing combined with edge-caching strategies to achieve median query response times of $T < 50\text{ms}$ for frequent requests.
- Resilient Architecture: Seamless automated fallbacks to local offline AI models when network connectivity is lost or cloud API timeouts occur.

6.0 End-to-End System Architecture Blueprint:

The platform follows a clean, decoupled micro services-ready layer

Layer	Technology Stack	Core Responsibilities
Presentation Tier	React Native / TailwindCSS	Cross-platform mobile UI, client-side contextual dashboard rendering, real-time query display.
API Gateway / Routing	Node.js / Express.js	Unified endpoint routing, JWT session authorization middleware, CORS control, and system rate limiting.
Caching Tier	Redis Enterprise 7.0	Caches active user interaction logs, immediate offline session tokens, and local prompt shortcuts (TTL 1h).
AI Engine Tier	Custom Edge AI / Cloud AI Orchestrator	Contextual natural language generation for hyper-personalized daily schedules and proactive productivity micro-routines.
External Data Tier	Core Device Telemetry / Device Sensors API	Fetches current geolocation metrics, user activity status, ambient sensor data, and system-level calendar feeds.
Persistence Tier	MongoDB Cluster 5.0	Securely stores encrypted user account configurations, device profiles, activity preference matrixes, and global search histories.

6.1 Data Modeling & User Domain Architecture

The core relational structure centers around two primary entities: Users and Connected Smart Device Profiles.

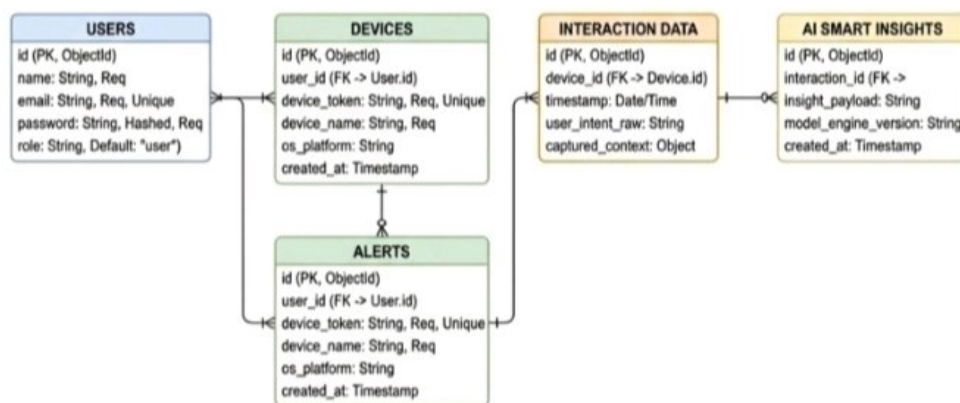
6.2 User Account Profile:

- Identity Constraints: Requires unique identifiers for client system usernames, verified system registration credentials, and linked primary device UUIDs.
- Credential Protection: Application passwords and private encryption keys are never stored as raw text; they undergo automatic salted standard cryptographic hashing prior to database persistence.
- Account Metadata: Automatically captures and logs account activation timestamps, device onboarding dates, and permission validation updates for audit trails.

6.3 Location Preferences:

- Relational Association: Binds each registered smart device or client application instance directly to a primary user account using secure database object relationships.
- Contextual Accuracy: Stores specific device hardware tokens, operating system profiles, global active permission configurations, and localized network nodes for micro-targeting smart automated alerts.

Entity-Relationship (ER) Diagram Description:



1. User Entity:

- User Entity
- id (Primary Key - Objectid)
- name (String, Required)
- email (String, Required, Unique)
- password (String, Hashed, Required)
- role (String, Default: "user")

2. Device Entity:

- id (Primary Key - ObjectId)

3. Interaction Data Entity:

- id (Primary Key - ObjectId)

4. AI Smart Insights Entity:

- id (Primary Key - ObjectId)

Key Structural Mappings:

- **User to Device (One-to-Many):** A registered user can manage and connect multiple favorite smart devices or client application instances.
- **Role-Based Route Guarding:** Strict token processing blocks unauthorized tiers from modifying core system configurations or accessing higher-level administrative APIs.
- **CRUD Execution Engine:** Complete schema management over interaction history, prompt templates, and AI insight assignment paths.
- **Categorization System:** Native tag indices facilitate quick filtering and querying of insights for enhanced personal discoverability.
- **Insight Generation Logic:** Protected entry points for managing and updating context verification flags (verified, pending, flagged).
- **Sanitization Safeguards:** Payload parameter sanitation prevents XSS vectors and NoSQL parameter manipulation injections across mobile and edge inputs.

Roles and Responsibilities

Admin:

- Responsible for user account management, including creating, deleting, or suspending client device profiles.
- Oversight of API gateway health, model latency metrics, and core system logs to monitor overall platform performance.
- Monitoring and managing database integrity, cloud synchronization pipelines, and system configurations.
 - Responsible for user account management, including creating, deleting, or suspending accounts.
 - Oversight of API health and system logs to monitor performance.
 - Monitoring and managing database integrity.

Registered User:

- Responsible for creating, updating, and maintaining a personalized list of favorite smart devices, custom automation routines, and context preferences.

Core System Script Implementations:

Application Entry Point (index.js):

The index.js file serves as the main entry point of the Pocket Smart AI application. It initializes the Express server, loads environment variables, configures middleware (such as rate limiters and session verification), establishes the MongoDB connection, registers API routes for smart interactions, and starts the server after a successful Database connection.

Epic4: Database Configuration

- API Testing Demo
- Configure MongoDB:
- Install Mongoose.
- In your Node.js project directory, run:
- `npm install mongoose`
- Create database connection.
- Create Schemas & Models.

Create Database connection:

Now, make sure the database is connected before performing any of the client actions through the backend infrastructure. The connection code looks similar to the one provided below.

Mongoose Schema Definitions

Location Schema:

SECTION 1: SYSTEM INITIALIZATION & ENVIRONMENT CONFIGURATION

```
// 1. Core Modules & Configuration
const express = require('express');
const mongoose = require('mongoose');
const dotenv = require('dotenv');
const cors = require('cors');

dotenv.config(); // Load variables
const app = express();
const PORT = process.env.PORT || 5000;
```

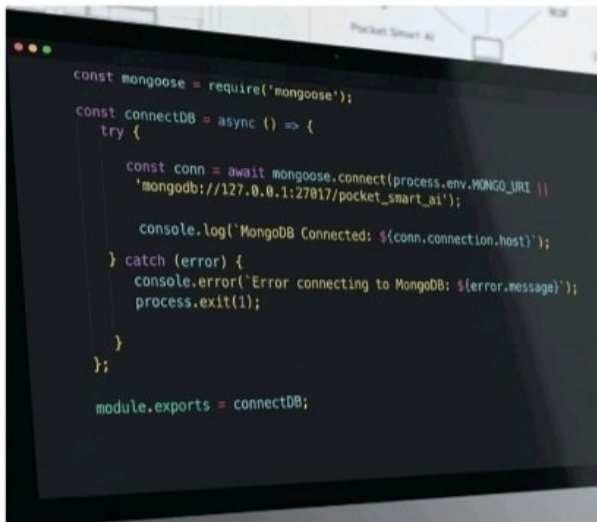
Metadata:

- Initializes the Express server.
- Loads environment variables (MongoDB URI, Port).

User schema:

Db connection:

This module establishes a connection between the application and MongoDB using Mongoose. It ensures that the backend architecture successfully establishes communication networks with the database layer before processing any active client requests.

A photograph of a computer screen displaying a code editor with JavaScript code for MongoDB connection. The code is as follows:

```
const mongoose = require('mongoose');
const connectDB = async () => {
  try {
    const conn = await mongoose.connect(process.env.MONGO_URI ||
    'mongodb://127.0.0.1:27017/pocket_smart_ai');

    console.log('MongoDB Connected: ${conn.connection.host}');
  } catch (error) {
    console.error('Error connecting to MongoDB: ${error.message}');
    process.exit(1);
  }
};

module.exports = connectDB;
```

Epic 5: Project Executions:

The Pocket Smart AI backend can be executed locally using Node.js. Before running the application, ensure that your MongoDB instance is running and that all required environment variables are configured correctly.

Running the Backend Server:

Install all required dependencies:

```
bash
npm install
```

Start the development server:

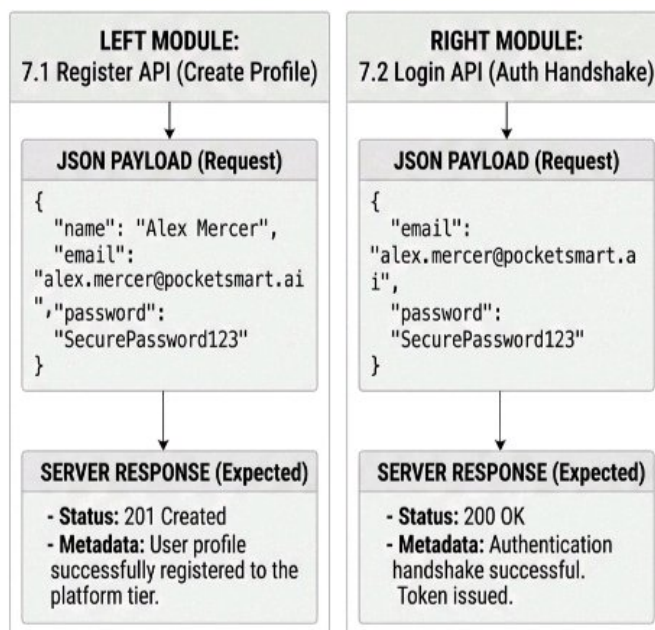
```
bash
npm start
```

After successful execution, the server connects to MongoDB and starts listening for Incoming API requests.

API testing:

Register API:

7.0 API Testing: User Authentication (Auth Handshake Flow)



Login API:

